Software Requirements Specification for Keystone: Capstone Project Management System

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

October 14, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 1
- 1.2 Scope . . . 1
- 1.3 Stakeholders . . . 3
- 1.4 Definitions and Acronyms . . . 3
- 1.5 Characteristics of Intended Reader . . . 5

2 System Overview **5**
- 2.1 Problem Statement . . . 5
- 2.2 Product Vision . . . 7
- 2.3 System Context . . . 8
- 2.4 User Characteristics . . . 10
- 2.5 System Constraints . . . 11

3 Stakeholder Requirements **11**

4 User Personas **11**
- 4.1 Persona 1 . . . 11
- 4.2 Persona 2 . . . 12

5 Use Cases **12**
- 5.1 UC1: Example Use Case . . . 12

6 Functional Requirements **13**

7 Data Requirements **13**
- 7.1 Major Data Entities . . . 13
- 7.2 Data Dictionary . . . 13
- 7.3 Privacy and Data Retention . . . 13

8 Non-Functional Requirements **13**

9 AI and Machine Learning Requirements **14**
- 9.1 Model Inputs . . . 14
- 9.2 Model Outputs . . . 14
- 9.3 Evaluation Metrics . . . 14
- 9.4 Known Limitations . . . 14

10 Requirements Traceability Matrices **14**

11 Rationale **15**

12 Likely Changes **15**

13 Unlikely Changes **15**

14 Appendices **16**
14.1 Appendix A: Additional Materials 16
14.2 Appendix B: Development Plan 16
14.3 Appendix C: Values of Auxiliary Constants 16
14.4 Appendix D: Generative AI Disclosure 16
14.5 Appendix E: Reflection 16

Revision History

Date	Version	Notes
Oct 13th 2026	1.0	Introduction and System Overview
Date 2	1.1	Notes

1 Introduction

This software requirements specification (SRS) document describes the system and product vision as well as the functional and non-functional requirements for Keystone. The SRS is intended to be a reference for the development team, stakeholders, and future maintainers of the system. This document is constructed using the IEEE 29148 and IEEE 15288 standards as a guide (**??**).

1.1 Purpose

The purpose of Keystone is to make capstone project proposals, discovery, team formation, and team project approval transparent, consistent, and easy to manage for the students, supervisors, and course instructors involved.

Capstone project and team selection currently happens across disconnected tools such as email, online forms, discussion forums, and shared files, leading to a lack of visibility, clarity, and detail in information available to students, supervisors, and instructors. As a result, project descriptions do not set clear expectations, students and supervisors cannot easily tell which projects are available or learn about each other, and instructors spend significant time manually coordinating the process. See Problem Statement for a more detailed description of the problem.

Keystone is intended to solve these problems by giving every participant a single, trustworthy source of information about capstone projects, teams, and their status, from the moment a project is proposed until it is approved.

1.2 Scope

Problem being solved. The fragmented and unreliable flow of information in capstone project and team selection and approval, as described in the Problem Statement section below.

Intended users. Capstone students, project supervisors (McMaster faculty and industry sponsors), course instructors, and teaching assistants (TAs), across the Software Engineering (SE), Computer Science (CS), and Mechatronics (TRON) capstone courses.

Major functionality. Keystone covers the lifecycle from project proposal to team project approval. Within that boundary the system:

- provides proposal templates for supervisors and for students, so that projects are described consistently;
- lets instructors review supervisors' project proposals before they are listed, and agree changes with the supervisors;
- provides a searchable catalog of projects showing which are open, taken, or able to take more teams;

- lets students and teams create structured profiles of their skills, experience, interests, and work style that other students and supervisors can view, indicate whether they are looking for a team, and find teammates interested in the same projects;
- suggests compatible teammates and suitable projects to students, based on their profiles and preferences
- lets student teams express interest in supervisor-proposed projects;
- supports communication between all users regarding project proposals, team formations, applications and and approvals, ensuring discussions are organized and accessible to relevant parties;
- supports comment threads on projects, student posts, and applications, so that questions and answers are kept in one place;
- lets supervisors and student teams each accept or decline working together, and records when both have agreed;
- notifies users of events that need their attention, such as new projects, status changes, and replies;
- provides supervisors with course process, timeline, and expectation information, and examples of past capstone projects, configurable by the course instructor;
- gives instructors visibility across courses into projects, teams, and their status, including which supervisors' projects were not selected, so that those supervisors can be informed;
- provides guidance and automation to help students meet all requirements when submitting their projects for approval, and when submitting changes to their project information;
- lets instructors approve, request changes to, or reject a team's project submission and later changes to its information;
- lets instructors configure which features are enabled and disabled for full flexibility

What Keystone does not do.

- It does not judge whether a project is feasible, sufficiently complex, or appropriate for a capstone course; this remains the responsibility of the course instructor.
- It does not make approval decisions. Final approval of teams and projects always rests with the course instructor. Courses may carry out approval inside the platform or in an external process.

- It does not resolve intellectual property (IP), confidentiality, or non-disclosure questions between students and supervisors.
- It does not find supervisors for student-proposed projects. Student-proposed projects are visible to all users of the platform, but Keystone provides no matching or request mechanism for supervisors.
- It does not schedule meetings or provide direct messaging between users. Meetings take place outside the platform, and discussion is limited to comment threads on posts and applications.
- It does not handle supervision after a project begins, such as feedback requests to supervisors, team performance concerns, or grading.

Expected inputs. Inputs are text-based project and profile information entered by users, and project status changes made by authorized users.

Expected benefits. Students can make more informed decisions on which projects to pursue, and find compatible teammates; supervisors know who is interested in their projects, learn about the students, and receive fewer repeated emails; and instructors spend less time on repetitive coordination and gain visibility into the whole process.

1.3 Stakeholders

- **Customer:** Course instructors for the SE, CS, and TRON capstone courses (Dr. Smith, Dr. Moradi, and Dr. Wassyng respectively). They decide which features their course uses and make the final project and team approval decisions.
- **End Users:**
  - Capstone students, who browse projects, create profiles, form teams, and apply to projects.
  - Project supervisors, who propose projects, manage their capacity, and review and confirm teams. Supervisors are either McMaster faculty or industry sponsors.
  - Course instructors and TAs, who manage users, projects, project status, and course configuration.
- **Development Team:** Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib.

1.4 Definitions and Acronyms

Term	Definition

Capstone course	A final-year design course in which student teams spend roughly eight months (September to April) designing, implementing, and validating a system for a real group of stakeholders. In this document, the SE, CS, and TRON capstone courses.
SE, CS, TRON	Software Engineering, Computer Science, and Mechatronics Engineering, the three programs in the Computing and Software Department whose capstone courses Keystone supports.
Course instructor	The instructor responsible for a capstone course and for approving its teams and projects.
Supervisor	A McMaster faculty member or industry sponsor who proposes a project, or who agrees to advise a team on a project, and provides domain knowledge or acts as a client.
Industry sponsor	A supervisor from outside McMaster, who may have no McMaster account (MacID).
Project proposal	A description of a project submitted by a supervisor, or a student, for teams to consider.
Student-proposed project	A project proposed by a student team rather than by a supervisor.
Project status	The position of a project in its lifecycle, for example proposed, listed, taken, or approved.
Student profile	A student's skills, experience, interests, work style, and project preferences, visible to other students and to supervisors.
Team profile	A description of a team's members and shared goals, project interests, and work style.
Supervisor capacity	The number of teams a supervisor is willing to supervise on a project.
Team formation rules	Course rules on team composition: which programs may work together on which projects, and the minimum and maximum team size.
Proposal approval	The course instructor's review of a supervisor's project proposal before it is listed for students. The instructor may list it as submitted, work with the supervisor to make changes, or reject it.
Team project approval	The course instructor's decision to approve a team's selected project and membership
Approval mode	How a course handles approval: in-platform, external, or integrated.
Custom document	A deliverable, chosen by each team from a list, that suits the type of project (for example, a usability testing report).

TA	Teaching assistant assigned to a capstone team after its project is approved.
MacID	A McMaster University user account identifier.

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

The intended reader of this SRS should have a background in software engineering, including knowledge of software development processes, requirements engineering, and system design principles. They do not have to be experts on these topics, but they should at least have the knowledge of a final year undergraduate student in software engineering. No knowledge of the capstone course process beyond the definitions in this section is required.

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

Current situation. Capstone team and project selection is spread across disconnected tools, including email, online forms, MS Teams, Avenue to Learn and GitLab. Supervisors submit proposals through a form. The SE course instructor, who collects proposals for all three courses, reviews them, discusses any changes with the supervisor by email, and edits the list by hand before it is distributed to students as a static list. Students then form teams informally, mainly through discussion forums on Avenue to Learn / Teams and personal contacts, contact supervisors by email, and submit their chosen project and team for instructor approval through a course-specific process. No single place shows the state of the whole process.

Problems. The following problems were identified through interviews with stakeholders and general observation.

- **Project descriptions are inconsistent and do not set expectations.** Proposals vary widely in the level of detail, and very short descriptions put students off. Some of the things students want to know include a project's end goal and expected impact. Also, descriptions often do not state how much of the project already exists. Supervisors are unsure what a capstone project should look like and what a proposal should include, and students who pitch their own ideas do not know what a proposal needs. This makes projects difficult to compare and leads to mismatched expectations.

- **Student-proposed projects are hard to see.** Students' own project ideas appear only in discussion forums, where supervisors cannot see them, including supervisors who would like to supervise a student-proposed project. They are not tracked alongside supervisor-proposed projects.

- **There is no visibility of project status.** Students and supervisors cannot see which projects are taken or open, or whether a supervisor is willing to take more than one team, and after a rejection students do not know which projects remain. New projects and updates are hard to notice, and users do not want to have to remember to check for them.

- **Compatible teammates are hard to find.** Students without an existing network rely on forum posts or chance. Teammates' project preferences, skills, and work styles are often unknown until a team has formed, so teams can disagree about what to build.

- **Supervisors do not know the students' backgrounds.** Supervisors have little structured information about the students and teams who contact them, such as their skills, experience, and program. They can only gain this information through interviews or rely on outside sources such as LinkedIn profiles, which are inconsistent and not always available.

- **Communication during selection is inefficient.** Supervisors receive repeated emails from teams, including after a team has been chosen. What one team learns about a project in a meeting is not shared with other teams applying for the same project. Nobody can see which teams are in contact with which supervisors. There is also no record that a team and a supervisor agreed to work together. Students sometimes report that a supervisor agreed when the supervisor has no record of the team, so the instructor must confirm agreements separately.

- **Supervisors lack guidance before proposing and responding.** Before proposing a project, supervisors receive only a short summary of the course's expectations and process. The full course information arrives only after teams and projects are settled, and supervisors want it earlier, along with examples of past projects. Supervisors are also unsure how quickly to reply to students and whether to speak to every team that contacts them. This leads to misunderstandings about what a capstone project should deliver and to many questions to the instructor.

- **Instructors lack visibility across courses.** Instructors cannot see which projects students in other capstone courses selected, so they cannot tell which proposed projects were not selected. Supervisors whose projects are not selected are not informed, which may discourage them from proposing projects in future years.

- **Instructor review is repetitive and time consuming.** Instructors review each team's submission manually and often give the same comments to many teams on simple items, such as missing information, wasting the instructor's time. When reviewing

project proposals, changes to proposals that are agreed with supervisors by email must then be applied to the project list by hand.

- **Records drift after approval.** Team and project information changes after approval (for example, titles, supervisors, links, team composition, and TA assignments), but updating the official record takes extra steps, so it is often not updated and the instructor's records become out of date.

Impact. Students lose time early in a project of limited duration, teams end up mismatched with their projects or with each other, supervisors may become reluctant to propose projects in later years, and course instructors carry the administrative burden of the process.

Desired improvement. All participants share a single, accurate and current source of information about projects, supervisors' capacity, teams, and approval status; students can find suitable projects and teammates; supervisors can see students' skills and ideas and receive information about the course when they need it; and course instructors keep full control of approval decisions while spending less time on repetitive tasks.

2.2 Product Vision

Keystone will be the central place where capstone projects are proposed, discovered, and matched with student teams, and where instructors can see and manage the process. Each course decides which features it uses and how its approval process is carried out, so the platform can be adopted by different courses, departments, and universities.

The goals of Keystone are:

1. **Consistent project information.** Every project, whether proposed by a supervisor or by students, is described in a consistent format that makes its objectives, expected impact, required skills, current state, and expected outcomes clear.

2. **Visibility of availability and updates.** Users can tell which projects are open, taken, or able to take additional teams, and are made aware of new projects and changes without having to check for them.

3. **Easier team formation.** Students can describe their skills, experience, work style, and project preferences in a structured profile, find teammates with compatible profiles and preferences, and propose projects of their own for others to join.

4. **Informed matching.** Students receive suggestions of compatible teammates and suitable projects based on their profiles and preferences.

5. **Informed supervisors.** Supervisors can see student-proposed projects and the skills and experience of the students and teams interested in their projects.

6. **Efficient communication during selection.** Questions and answers about a project or application are kept in one place, supervisors can see which teams are interested in their projects, and fewer repeated emails are needed.

7. **Clear agreement between supervisors and teams.** A supervisor's and a team's agreement to work together is recorded in the platform and is visible to the instructor.

8. **Timely guidance for supervisors.** Supervisors have access to information about the capstone course, its timeline, and its expectations, and to examples of past projects, when they consider proposing a project, as configured by the course instructor.

9. **Oversight for instructors.** Instructors can see projects, teams, supervisors, and their status across courses, including projects that were not selected.

10. **Instructor-controlled approval with less repetitive work.** Final approval decisions remain with the course instructor. The platform supports the project proposal approval process, guiding supervisors to write more detailed project proposals and facilitating discussion between the instructor and supervisor. The platform supports the team project approval process carried out in the course, whether that takes place in the platform or elsewhere, helps students submit complete and correct information, and helps keep records current.

11. **Configurable and reusable.** Features can be turned on or off, and course-specific rules can be configured, so that the platform serves different courses without modification.

2.3 System Context

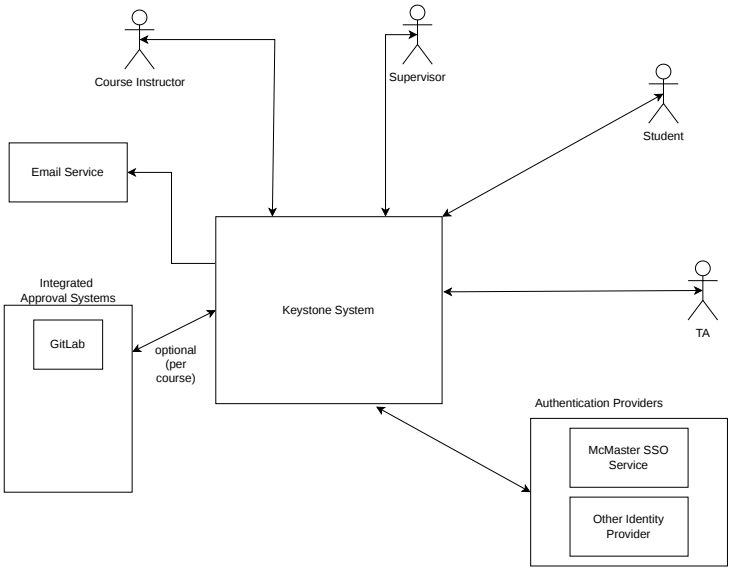


Figure 1: System Context

The entities that interact with Keystone are students, supervisors, course instructors, TAs, authentication providers, an email service, and the integrated approval systems of courses that do not carry out approval within Keystone.

- Student Responsibilities:
  - Provide an accurate profile including skills, experience, interests, work style, and project preferences.
  - Provide accurate team information and project proposals, including student-proposed projects.
  - Express interest in projects, take part in discussions, and respond to supervisors' decisions.
  - Submit complete project and team information for approval and keep it current.
- Supervisor Responsibilities:
  - Provide accurate, complete project proposals, including the current state of the project, expected outcomes, and the number of teams they are willing to supervise.
  - Review interested teams, take part in discussions, and accept or decline working with each team.
  - Keep their capacity and project information current.
- Course Instructor Responsibilities:
  - Configure the features, rules, and approval mode that apply to their course.
  - Make final decisions on teams and projects, including project proposal and team project approvals, requests for changes, and rejection.
  - Judge project feasibility and appropriateness for the course.
  - Provide the course process, timeline, and expectation information and the examples of past projects shown to supervisors.
- TA Responsibilities:
  - Update project and team status as directed by instructors, in courses where approval takes place outside Keystone.
  - Assist instructors in keeping records accurate.
- Authentication Provider Responsibilities:
  - Verify the identity of users who sign in, including users with and without McMaster accounts.
- Email Service Responsibilities:

  - Deliver the notifications generated by Keystone to users' email addresses.

- Integrated Approval System Responsibilities:
  - In courses that use one, carry out approval of teams and projects and make the outcome available to Keystone. Otherwise, instructors or TAs record the outcome in Keystone manually.

- Keystone Responsibilities:
  - Provide proposal templates, and check that required project, profile, and team information is present and well formed.
  - Show current project status, availability, and supervisor capacity.
  - Apply the team formation rules configured for each course.
  - Keep discussions attached to the project, post, or application they concern, and record whether a supervisor and a team have agreed to work together.
  - Restrict what each user can see and change according to their role.
  - Record who made each change to a project or team, and when.
  - Notify users of events that need their attention.
  - Provide instructors with an overview of projects, teams, and their status across courses.
  - Suggest compatible teammates and projects based on profile data.

Keystone is used to coordinate an academic process. Use is concentrated around the start of the academic year, when projects are proposed and teams are formed, with lighter use afterwards. It is not mission-critical or safety-critical. A failure delays the coordination of projects and teams but does not cause harm. The data handled includes personal information about students, so privacy is the main risk to be managed, rather than safety.

2.4 User Characteristics

- **Students** are final-year undergraduate students in the SE, CS and TRON programs. They are comfortable using web applications and should be able to figure out how to use Keystone without any training. They use Keystone mainly at the start of the course. They are motivated to find a suitable project quickly, but differ in whether they already have a team or a project idea.

- **Supervisors** are McMaster faculty or industry sponsors. They have a wide range of backgrounds. They may use Keystone only a few times a year, have limited time, and may be unfamiliar with the capstone course, its expectations, and its timeline. Industry sponsors may have no McMaster account. No technical training beyond using a web application can be assumed, and the effort needed to propose a project must stay low enough that it does not discourage them.

- **Course instructors** know the course's process and rules. They configure the platform for their course and make approval decisions. They use it throughout the term, and need to configure it without developer help.
- **TAs** are graduate students who know the course's process. They help instructors with day-to-day administration and are comfortable using web applications and technical tools.

2.5 System Constraints

- **Web-based.** Keystone shall be accessible through current versions of common web browsers on desktop, tablet, and mobile devices, and shall not require specialized hardware or installation. An internet connection is required.
- **Modular features.** Each major feature shall be able to be enabled or disabled independently for each course, so that a course can use only the parts that fit its process.
- **Independent of any one approval tool.** Keystone shall not depend on a specific external tool, such as the GitLab merge request process used by the Software Engineering course, to function. It shall support courses that carry out approval in the platform, courses that carry it out elsewhere, and, where an adapter exists, courses that synchronize with an external process.
- **Users outside the university.** Keystone shall support users who do not have a McMaster account (such as industry sponsors) alongside users who do.
- **Configurable course rules.** As Keystone should be extensible to different capstone courses, course-specific rules, such as which programs may work together on which projects and the minimum and maximum team size, shall be set by the course instructor as configuration and shall not be fixed in the design.

3 Stakeholder Requirements

SR1 TODO

SR2 TODO

SR3 TODO

4 User Personas

4.1 Persona 1

- Name:

- Description:
- Goals:
- Frustrations:
- Technical Ability:

4.2 Persona 2

- Name:
- Description:
- Goals:
- Frustrations:
- Technical Ability:

5 Use Cases

5.1 UC1: Example Use Case

Primary Actor

TODO

Preconditions

- TODO

Main Success Scenario

1. TODO
2. TODO
3. TODO

Alternative Flows

- TODO

Postconditions

- TODO

6 Functional Requirements

FR1: The system shall ...

FR2: The system shall ...

FR3: The system shall ...

7 Data Requirements

7.1 Major Data Entities

- User
- Song
- Session
- Report

7.2 Data Dictionary

Attribute	Type	Description
user_id	UUID	Unique identifier
email	String	User email address

Table 2: Data Dictionary

7.3 Privacy and Data Retention

TODO

8 Non-Functional Requirements

NFR1: **Accuracy**

NFR2: **Usability**

NFR3: **Maintainability**

NFR4: **Portability**

- Other NFRs that might be discussed include verifiability, understandability, reusability, performance, reliability and availability, security, and accessibility. You may have to add other NFRs.

9 AI and Machine Learning Requirements

9.1 Model Inputs

TODO

9.2 Model Outputs

TODO

9.3 Evaluation Metrics

TODO

9.4 Known Limitations

TODO

10 Requirements Traceability Matrices

Stakeholder Req	Use Case	Functional Req
SR1	UC1	FR1

Table 3: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR1	NFR1

Table 4: Traceability Between Functional and Nonfunctional Requirements

11 Rationale

12 Likely Changes

LC1:

13 Unlikely Changes

LC2:

14 Appendices

14.1 Appendix A: Additional Materials

TODO

14.2 Appendix B: Development Plan

14.3 Appendix C: Values of Auxiliary Constants

TODO

14.4 Appendix D: Generative AI Disclosure

14.5 Appendix E: Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be

related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?